CIS613 Assignment - Mutation Testing - Ahsan

Ahsanul Kabir

April 2026

1 Premise

For the 'Unit Testing Frameworks' assignment, I initially used Java. However, for this assignment, I chose Python to get used to its unit testing. My previous test cases were not organized based on testing types, so I decided to start fresh and structure my tests more systematically this time.

I modified the given *triangle.py* to add explicit boundary value testing. Each variable's bounds are from 1 to 200, inclusive.

Versions used:

- Python: 3.10.20
- Mutmut: 2.5.0

I created three separate test files. They are:

- Boundary Value Analysis (BVA): Tests were created using the Normal Boundary Value Analysis principle. Each variable takes the boundary values 1, 200, and one greater than the lowest 1 and one less than the highest 200. At one time, one variable is changed.
- Equivalence Class (EC) Testing: Weak Normal Equivalence Class Testing based on the Output Classes. We have four output classes: EQUILATERAL, ISOSCELES, SCALENE, and INVALID. One test is written for each class.
- Decision Table (DT) Based Testing: Decision table used for test cases:

	Rule 1	**Rule 2**	**Rule 3**	**Rule 4**	**Rule 5**	**Rule 6**	**Rule 7**
Conditions							
C1: All sides in bounds ($1 \leq$ side ≤ 200)?	F	T	T	T	T	T	T
C2: Triangle inequality met?	-	F	T	T	T	T	T
C3: $a == b$?	-	-	T	T	F	F	F
C4: $b == c$?	-	-	T	F	T	F	F
C5: $a == c$?	-	-	T	F	F	T	F
Actions							
Return `INVALID`	X	X					
Return `EQUILATERAL`			X				
Return `ISOSCELES`				X	X	X	
Return `SCALENE`							X

Table 1: Decision Table for Triangle Identifier

2 Code Coverage

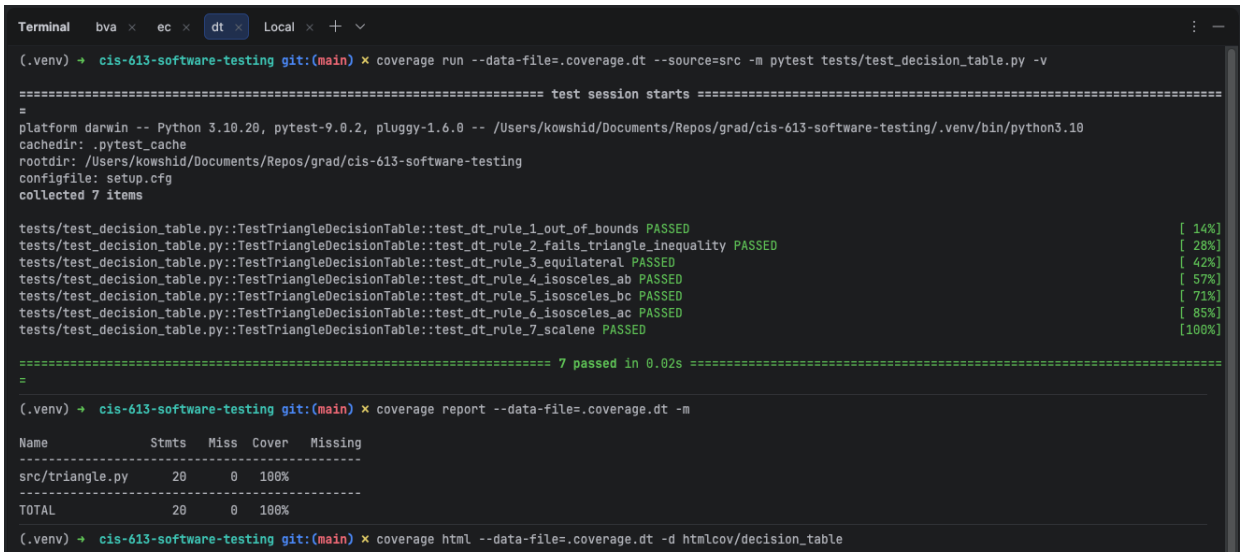
- Boundary Value Analysis (BVA): 90%
- Equivalence Class (EC) Testing: 95%
- Decision Table (DT) Based Testing: 100%
- Unit Testing Framework: 100%

```
Terminal   bva   ec   dt   Local

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage run --data-file=.coverage.bva --source=src -m pytest tests/test_bva.py -v

============================================================================= test session starts ==============================================================================
platform darwin -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0 -- /Users/kowshid/Documents/Repos/grad/cis-613-software-testing/.venv/bin/python3.10
cachedir: .pytest_cache
rootdir: /Users/kowshid/Documents/Repos/grad/cis-613-software-testing
configfile: setup.cfg
collected 4 items

tests/test_bva.py::TestTriangleBVA::test_is_valid_bounds_boundary_value_covered PASSED                                                                                    [ 25%]
tests/test_bva.py::TestTriangleBVA::test_normal_bva_a_varies PASSED                                                                                                       [ 50%]
tests/test_bva.py::TestTriangleBVA::test_normal_bva_b_varies PASSED                                                                                                       [ 75%]
tests/test_bva.py::TestTriangleBVA::test_normal_bva_c_varies PASSED                                                                                                       [100%]

============================================================================== 4 passed in 0.02s ===============================================================================

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage report --data-file=.coverage.bva -m

Name              Stmts   Miss  Cover   Missing
-----------------------------------------------
src/triangle.py      20      2    90%   11, 22
-----------------------------------------------
TOTAL                20      2    90%

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage html --data-file=.coverage.bva -d htmlcov/bva

Wrote HTML report to htmlcov/bva/index.html
```

Figure 1: Boundary Value Analysis Code Coverage

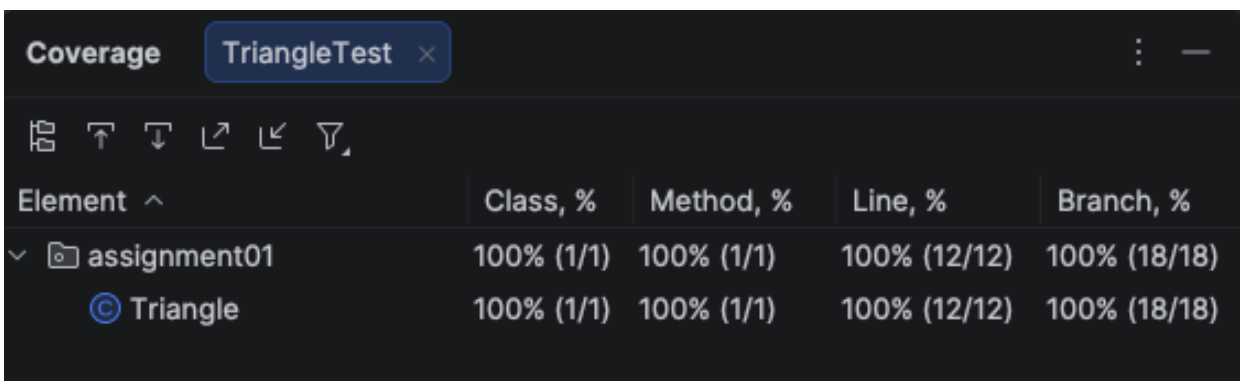
```
Terminal   bva   ec   dt   Local

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage run --data-file=.coverage.ec --source=src -m pytest tests/test_equivalence_class.py -v

============================================================================= test session starts ==============================================================================
platform darwin -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0 -- /Users/kowshid/Documents/Repos/grad/cis-613-software-testing/.venv/bin/python3.10
cachedir: .pytest_cache
rootdir: /Users/kowshid/Documents/Repos/grad/cis-613-software-testing
configfile: setup.cfg
collected 1 item

tests/test_equivalence_class.py::TestTriangleEquivalenceClass::test_weak_normal_equivalence_class PASSED                                                                  [100%]

============================================================================== 1 passed in 0.01s ===============================================================================

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage report --data-file=.coverage.ec -m

Name              Stmts   Miss  Cover   Missing
-----------------------------------------------
src/triangle.py      20      1    95%   11
-----------------------------------------------
TOTAL                20      1    95%

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage html --data-file=.coverage.ec -d htmlcov/equivalence_class

Wrote HTML report to htmlcov/equivalence_class/index.html
```

Figure 2: Equivalence Class Testing Code Coverage



```
Terminal   bva   ec   dt   Local

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage run --data-file=.coverage.dt --source=src -m pytest tests/test_decision_table.py -v

============================================================================= test session starts ==============================================================================
platform darwin -- Python 3.10.20, pytest-9.0.2, pluggy-1.6.0 -- /Users/kowshid/Documents/Repos/grad/cis-613-software-testing/.venv/bin/python3.10
cachedir: .pytest_cache
rootdir: /Users/kowshid/Documents/Repos/grad/cis-613-software-testing
configfile: setup.cfg
collected 7 items

tests/test_decision_table.py::TestTriangleDecisionTable::test_dt_rule_1_out_of_bounds PASSED                                                                          [ 14%]
tests/test_decision_table.py::TestTriangleDecisionTable::test_dt_rule_2_fails_triangle_inequality PASSED                                                              [ 28%]
tests/test_decision_table.py::TestTriangleDecisionTable::test_dt_rule_3_equilateral PASSED                                                                            [ 42%]
tests/test_decision_table.py::TestTriangleDecisionTable::test_dt_rule_4_isosceles_ab PASSED                                                                           [ 57%]
tests/test_decision_table.py::TestTriangleDecisionTable::test_dt_rule_5_isosceles_bc PASSED                                                                           [ 71%]
tests/test_decision_table.py::TestTriangleDecisionTable::test_dt_rule_6_isosceles_ac PASSED                                                                           [ 85%]
tests/test_decision_table.py::TestTriangleDecisionTable::test_dt_rule_7_scalene PASSED                                                                                [100%]

============================================================================== 7 passed in 0.02s ===============================================================================

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage report --data-file=.coverage.dt -m

Name              Stmts   Miss  Cover   Missing
-----------------------------------------------
src/triangle.py      20      0   100%
-----------------------------------------------
TOTAL                20      0   100%

(.venv) ➜  cis-613-software-testing git:(main) ✗ coverage html --data-file=.coverage.dt -d htmlcov/decision_table
```

Figure 3: Decision Table-Based Testing Code Coverage



Coverage | TriangleTest

Element	Class, %	Method, %	Line, %	Branch, %
assignment01	100% (1/1)	100% (1/1)	100% (12/12)	100% (18/18)
Triangle	100% (1/1)	100% (1/1)	100% (12/12)	100% (18/18)

Figure 4: Unit Testing Framework Code Coverage

3 Mutation Scores

Formula: $MutationScore = KilledMutants/TotalMutants$

- Normal Boundary Value Analysis: $32/51 = 0.63$
- Equivalence Class Testing: $21/51 = 0.41$
- Decision Table-Based Testing: $22/51 = 0.43$
- Previous unit testing: $38/41 = 0.93$

4 Analysis

4.1 Boundary Value Effect

BVA specifically targets the edges of the input domains. Many of the mutants generated by mutmut are off-by-one errors or boundary shifts, like changing <= to <. Because my BVA suite explicitly tests values exactly on the boundaries $[1, 200]$ and just outside them, it successfully caught mutants that shifted these boundaries.

Equivalence Class (EC) testing relies on picking representative values from broad categories. It does not aggressively test the edges. Because these values are comfortably in the middle of the $[1, 200]$ range, they are completely immune to boundary mutations. The same is true for Decision Tables (DT) based testing. Decision tables are used to ensure all logical combinations of conditions are met. To satisfy these logical rules, I used mid-range arbitrary numbers.

In both cases, similar mutants survive because EC and DT methods never probe the edges where these mutations take effect. The only mutant killed by the DT method but not killed by the EC is the one where boundary checking is changed to 201 from 200.

4.2 The Constant Mutants Effect

Mutmut altered constant values in the source code. For example, changing INVALID = -1 to INVALID = 1 or None. These mutations survived in all the methods because the test cases imported the same constants directly from the source file $src.triangle$. As a result, when a constant was mutated, the tests also used the updated value, causing them to still pass even though the logic had been altered.

4.3 The None/EmptyString/False Mutant

For some mutants, the variable $is_a_triangle$ changed between $False/None/$"". This mutation survived because, in Python, both None and an empty string evaluate to False in boolean contexts, making them functionally equivalent in this scenario.

4.4 Old Java Code vs Latest Codes

I modified the source code, and the tools used to get the mutation score are different between the old unit test framework tests and the new test suites. Thus, comparing them directly is not possible. One pattern I noticed is that previous unit testing has a very high mutation score, but it does not fail where the new test suites fail the most: the constant mutant effect. Test codes in both languages (Java and Python) are using the constants from the same source code.

5 Comparison with Other Test Variants

For boundary value testing, the $MutationScore$ would increase if Robust Worst-Case Boundary Value Testing is used. In that case, the mutants that change the triangle side max value to 200 from 201 would be caught. An example can be found in mutant 25:

```
# mutant 25
--- src/triangle.py
+++ src/triangle.py
@@ -7,7 +7,7 @@
 def triangle_identifier(a, b, c):
     is_a_triangle = None

-    if not (1 <= a <= 200) or not (1 <= b <= 200) or not (1 <= c <= 200):
+    if not (1 <= a <= 200) or not (1 <= b <= 200) or not (1 <= c <= 201):
         is_a_triangle = False
     elif (a < b + c) and (b < a + c) and (c < a + b):
         is_a_triangle = True
```

If there were test cases that failed 201 value due to boundary exceeding, then that would have been addressed through mutation.

6 Addressing More Mutants

One way to address more mutants is to define the same constants in each test file rather than importing them from the source file. In that case, *The Constant Mutants* would be modified in the source code, but not in the test suites.

For this, I created $test_combined.py$ and combined all the test codes from the three suites together. I ran mutation testing and got a score of $34/51 = 0.67$ without any code modification other than compiling the codes together. I then added the constants in the Python test file and ran the mutation test again. This time the mutation score was $43/51 = 0.83$.

This would add value to the test suite as we can identify any potential constant changes right away.